Primeiras simulações em grupo de Teoria da Informação

Como sempre: DRAFT
(cuidado com erros!)

2026

Sumário

1 Tópico 1: Entropia, Informação Mútua e o Teste de Independência **2**
- 1.1 Bloco 1: Conceitos principais . . . 2
- 1.2 Bloco 2: Roteiro de simulação . . . 4
- 1.3 Bloco 3: Sugestões de implementação em Python (adaptem se preferirem R) . . . 5

2 Tópico 2: Desigualdade de Processamento de Dados e Cadeias de Markov **5**
- 2.1 Bloco 1: Conceitos principais . . . 6
- 2.2 Bloco 2: Roteiro de simulação . . . 6
- 2.3 Bloco 3: Sugestões de implementação em Python . . . 7

3 Tópico 3: Compressão de Dados, AEP e o Critério de Kelly **8**
- 3.1 Bloco 1: Conceitos principais . . . 8
- 3.2 Bloco 2: Roteiro de simulação . . . 9
- 3.3 Bloco 3: Sugestões de implementação em Python . . . 10

4 Tópico 4: Capacidade de Canal e Códigos Corretores de Erro **11**
- 4.1 Bloco 1: Conceitos principais . . . 11
- 4.2 Bloco 2: Roteiro de simulação . . . 12
- 4.3 Bloco 3: Sugestões de implementação em Python . . . 13

Introdução

Este material apresenta quatro atividades de simulação para serem desenvolvidas em grupos de 5 a 6 estudantes do curso, usando Python, R ou outra linguagem de programação de livre escolha do grupo. Cada atividade gira em torno de um problema concreto, testando com dados gerados por vocês mesmos, previsões que a teoria da informação faz sobre esse problema.

Para cada tópico, o material está dividido em três blocos.

O primeiro bloco apresenta os conceitos teóricos centrais da atividade, com uma explicação intuitiva do que cada quantidade mede e por que ela importa. Não é uma revisão completa da teoria, isso já foi visto em aula, mas um lembrete direcionado ao que será usado na simulação.

O segundo bloco propõe um roteiro de simulação: uma sequência de passos no estilo de um protocolo de laboratório. Os passos descrevem o que fazer, não como programar. Cada grupo decide livremente a forma exata de implementar cada passo.

O terceiro bloco traz sugestões mais concretas sobre a implementação em Python, incluindo estruturas de dados, funções úteis e armadilhas comuns. Mesmo assim, nenhum código pronto é fornecido. A ideia é que vocês discutam entre si e com assistentes de IA (Claude, ChatGPT, Gemini, Copilot, ou qualquer outro) até chegar ao código final, rodem os experimentos, e escrevam, para cada tópico, um relatório curto descrevendo o que foi feito, os principais resultados numéricos ou gráficos obtidos, e se as previsões teóricas se confirmaram ou não.

Evitem, sempre que possível, resolver a atividade chamando uma única função pronta de alguma biblioteca que calcula a quantidade de interesse sem vocês verem o que está acontecendo por dentro. O objetivo destas atividades não é obter um número certo o mais rápido possível. É entender de onde aquele número vem. Gerem as sequências aleatórias com as regras explícitas do modelo que vocês definiram, contem frequências "na mão" a partir dos dados simulados, simulem a passagem de bits por um canal ruidoso bit a bit quando for o caso. Bibliotecas prontas podem, e devem, ser usadas depois, como conferência independente de um resultado que vocês já calcularam por conta própria, ou em tarefas auxiliares que não são o núcleo conceitual do exercício.

Uma observação metodológica: qualquer quantidade estimada a partir de uma amostra finita (uma entropia, uma informação mútua, uma probabilidade) carrega um erro que depende do tamanho da amostra e, no caso da entropia e da informação mútua, também do tamanho do alfabeto envolvido. Amostras pequenas tendem a produzir estimativas de informação mútua sistematicamente maiores do que o valor verdadeiro, mesmo quando as variáveis são de fato independentes. Fiquem atentos a esse viés ao interpretar os resultados, principalmente nos passos que envolvem testes de hipótese.

1 Tópico 1: Entropia, Informação Mútua e o Teste de Independência

O cenário

Uma rede de postos de saúde acompanha pacientes com suspeita de dengue usando um teste rápido de antígeno. Os resultados dos testes (positivo ou negativo) são anotados no prontuário de cada paciente. Uma vez por semana, cada posto precisa transmitir todo o histórico de testes através de um link de rádio via satélite, caro e trabalhoso.

A equipe responsável pelo sistema tem duas perguntas bem práticas antes de investir na infraestrutura de transmissão. Primeira: o teste rápido realmente carrega informação sobre a condição de saúde verdadeira do paciente, ou seria só ruído caro de transmitir? Segunda: quantos bits, no mínimo, são necessários para enviar o histórico de n testes de um paciente, aproveitando o fato de que o estado de saúde de um teste tende a se parecer com o do teste anterior?

1.1 Bloco 1: Conceitos principais

Sejam D o estado real de saúde do paciente ($D = 1$ se infectado, $D = 0$ caso contrário) e T o resultado do teste rápido ($T = 1$ positivo, $T = 0$ negativo), duas variáveis aleatórias discretas com distribuição conjunta $p(d, t)$.

Entropia. A quantidade $H(X) = -\sum_x p(x) \log_2 p(x)$ mede, em bits, a incerteza média sobre o valor de uma variável aleatória, ou equivalentemente o número médio de bits necessários para descrevê-la da forma mais eficiente possível. Quanto mais previsível a variável, menor a entropia.

Entropia conjunta e condicional. A entropia conjunta $H(D, T)$ mede a incerteza sobre o par (D, T) como um todo. A entropia condicional $H(D \mid T) = H(D, T) - H(T)$ mede quanta

incerteza sobre D ainda resta depois de sabermos T. Faz sentido que $H(D \mid T) \leq H(D)$: saber o resultado do teste nunca aumenta, em média, a incerteza sobre o estado real de saúde.

Entropia cruzada e entropia relativa. Suponham que a equipe assuma, para projetar um código de transmissão, uma distribuição $q(d)$ para o estado de saúde que não é exatamente a verdadeira $p(d)$, talvez baseada em dados de outra região. O custo médio real de transmitir D usando um código otimizado para q é a entropia cruzada

$$H(p, q) = -\sum_{d} p(d) \log_2 q(d).$$

O excesso de bits pago por usar o modelo errado q em vez do verdadeiro p é a entropia relativa, também chamada divergência de Kullback-Leibler,

$$D(p\|q) = \sum_{d} p(d) \log_2 \frac{p(d)}{q(d)} = H(p, q) - H(p), \tag{1}$$

sempre maior ou igual a zero, e igual a zero apenas quando $q = p$. Como vimos $D(p\|q)$ não é simétrica: o custo de confundir p com q não é, em geral, igual ao custo de confundir q com p.

Informação mútua. A quantidade central desta atividade é

$$I(D; T) = \sum_{d,t} p(d, t) \log_2 \frac{p(d, t)}{p(d)p(t)} = H(D) - H(D \mid T) = H(T) - H(T \mid D), \tag{2}$$

que mede quantos bits, em média, aprendemos sobre D ao observar T, e, de forma talvez algo surpreendente, exatamente os mesmos bits que aprendemos sobre T ao observar D, apesar de a definição parecer assimétrica. Se $I(D; T) = 0$, o teste é estatisticamente inútil: o resultado não diz nada sobre a condição real do paciente. Quanto maior $I(D; T)$, mais informativo é o teste.

Informação mútua condicional. Se uma terceira variável Z está disponível, por exemplo a idade do paciente ou se ele já teve dengue antes, a quantidade

$$I(D; T \mid Z) = H(D \mid Z) - H(D \mid T, Z)$$

mede quanta informação T ainda carrega sobre D depois de descontar o que Z já revela.

Regra da cadeia. Para uma sequência de variáveis aleatórias $X^n = (X_1, \ldots, X_n)$, a entropia conjunta se decompõe como

$$H(X^n) = \sum_{i=1}^{n} H(X_i \mid X^{i-1}), \qquad X^{i-1} = (X_1, \ldots, X_{i-1}), \tag{3}$$

ou seja, a incerteza total da sequência inteira é a soma das incertezas de cada novo elemento condicionada em tudo o que veio antes. Essa regra é o motivo matemático pelo qual comprimir uma sequência correlacionada elemento a elemento, usando o passado como contexto, pode custar muito menos bits do que comprimir cada elemento isoladamente, ideia que vocês vão explorar diretamente no roteiro abaixo com o histórico de testes de vários dias.

Teste de independência. Como decidir, a partir de dados, se D e T são realmente dependentes, ou se a dependência aparente é só fruto do acaso de uma amostra finita? Sob a hipótese nula H_0 de que D e T são independentes, a estatística

$$2N \ln(2)\, \hat{I}(D; T) \xrightarrow{d} \chi^2_{(|\mathcal{D}|-1)(|\mathcal{T}|-1)} \tag{4}$$

converge em distribuição para uma qui-quadrado com $(|\mathcal{D}| - 1)(|\mathcal{T}| - 1)$ graus de liberdade, em que N é o tamanho da amostra e $\hat{I}(D; T)$ é a informação mútua estimada, em bits, a partir das frequências observadas.

Essa é essencialmente o teste usado classicamente em tabelas de contingência, apenas reescrito em termos de informação mútua. Valores grandes da estatística, equivalentemente p-valores pequenos, levam à rejeição da independência.

1.2 Bloco 2: Roteiro de simulação

Passo 1: Definam o modelo probabilístico completo que vai servir de verdade na simulação: a probabilidade $p(D = 1)$ de infecção na população acompanhada, a taxa de falso positivo $p(T = 1 \mid D = 0)$ e a taxa de falso negativo $p(T = 0 \mid D = 1)$ do teste rápido. Calculem, a partir desse modelo, os valores teóricos de $H(D)$, $H(T)$, $H(D, T)$, $H(D \mid T)$ e $I(D; T)$.

Passo 2: Gerem uma amostra de N pares (D, T) simulando esse modelo, representando N pacientes observados uma única vez cada um.

Passo 3: A partir das frequências observadas na amostra, estimem $H(D)$, $H(T)$, $H(D, T)$, $H(D \mid T)$ e $I(D; T)$, e comparem cada uma com o valor teórico do Passo 1.

Passo 4: Repitam a estimativa de $I(D; T)$ para vários tamanhos de amostra crescentes, por exemplo $N = 20$, 50, 100, 500, 2000, 10000, e observem como o erro entre a estimativa e o valor teórico se comporta.

Passo 5: Escolham uma distribuição adivinhada $q(D)$, possivelmente diferente da verdadeira $p(D)$ definida no Passo 1, e calculem a entropia cruzada $H(p, q)$ e a divergência $D(p\|q)$. Confirmem que a diferença entre o custo de transmitir usando o modelo errado e o custo ótimo bate com $D(p\|q)$.

Passo 6: Construam agora uma dimensão temporal: simulem o histórico de testes de um único paciente ao longo de n dias, $T_1, \ldots, T_n$, em que o estado de saúde subjacente tem alguma persistência de um dia para o outro, isto é, T_i depende de T_{i-1}. Uma cadeia de Markov de dois estados é suficiente para gerar essa dependência.

Passo 7: Estimem a entropia da sequência completa $H(T_1, \ldots, T_n)$ e comparem com a soma das entropias condicionais dada pela regra da cadeia, e também com o valor ingênuo $n \cdot H(T_1)$, que seria o custo de transmitir cada dia de forma independente, sem explorar a dependência temporal.

Passo 8: Quantifiquem, em bits, quanto se economiza transmitindo o histórico completo de forma condicional, explorando a dependência entre dias consecutivos, em vez de transmitir cada dia isoladamente. Essa economia está diretamente relacionada à informação mútua entre dias vizinhos.

Passo 9: Apliquem o teste de independência descrito no Bloco 1 para decidir, a partir de uma amostra simulada, se D e T são de fato dependentes, e separadamente se T_i e T_{i-1} são dependentes. Reportem a estatística calculada, os graus de liberdade e a conclusão.

Passo 10: Repitam o experimento completo em três cenários (dependência forte, dependência fraca, independência real entre D e T) e verifiquem, rodando cada cenário várias vezes, se o teste do Passo 9 acerta a decisão na maioria das repetições. Quantas vezes o teste rejeita a independência quando ela é, de fato, verdadeira?

1.3 Bloco 3: Sugestões de implementação em Python (adaptem se preferirem R)

Construam funções pequenas e reaproveitáveis: uma para entropia a partir de um vetor de probabilidades ou de contagens, normalizando internamente, uma para entropia conjunta a partir de uma tabela bidimensional de probabilidades ou contagens, e uma para informação mútua a partir dessa mesma tabela conjunta. Definir essas funções uma vez, no início do notebook ou do script, e reutilizá-las em todos os passos evita erros de copiar e colar e vai facilitar a vida nos outros três tópicos também, já que as mesmas quantidades reaparecem o tempo todo.

Para gerar a amostra do Passo 2, o módulo `numpy.random` tem tudo que vocês precisam: sorteiem primeiro D com a probabilidade definida, depois sorteiem T condicionalmente ao valor de D obtido, usando as taxas de falso positivo e falso negativo.

Para contar frequências e montar a tabela conjunta empírica, um histograma bidimensional simples resolve bem, por exemplo `numpy.histogram2d`, ou contando ocorrências de cada combinação, já que aqui o alfabeto é pequeno, dois valores para cada variável. Ao calcular logaritmos de probabilidades estimadas, lembrem de tratar contagens zero separadamente. O termo correspondente na soma da entropia é simplesmente descartado quando $p = 0$, pois $\lim_{p \to 0^+} p \log p = 0$, evitando erros numéricos ou valores indefinidos.

A cadeia de Markov do Passo 6 pode ser implementada com um laço simples que, a cada dia, sorteia o novo estado a partir de uma matriz de transição 2×2 e do estado do dia anterior. Não há necessidade de bibliotecas especializadas em cadeias de Markov para um exemplo de dois estados, o laço direto é mais transparente e mais fácil de depurar.

Para a estatística do teste de independência do Passo 9, depois de calcularem $2N \ln(2)\, \hat{I}$ na mão a partir da tabela de contagens, usem `scipy.stats.chi2` para obter o valor p associado aos graus de liberdade corretos. A função `scipy.stats.chi2_contingency`, com a opção que seleciona a razão de verossimilhanças em vez da estatística de Pearson (consultem a documentação do parâmetro que controla qual membro da família de estatísticas de divergência é usado), deve devolver exatamente o mesmo valor que vocês calcularam a partir da informação mútua. Se não bater, o problema está em algum lugar da conta de vocês, não da biblioteca.

No Passo 10, ao repetir o experimento inteiro muitas vezes para estudar a taxa de erro do teste, organizem os resultados de cada repetição, a estatística calculada, o valor p e a decisão, em um vetor ou em um `DataFrame` do `pandas`, para depois calcular facilmente a fração de rejeições em cada cenário. Um lembrete de bom senso experimental: com contagens esperadas muito pequenas em alguma célula da tabela de contingência, regra prática, menos de cinco, a aproximação qui-quadrado deixa de ser confiável. Usem tamanhos de amostra N generosos o suficiente para evitar essa armadilha, e comentem no relatório se perceberam algum efeito disso.

2 Tópico 2: Desigualdade de Processamento de Dados e Cadeias de Markov

O cenário

Vocês já devem ter brincado de telefone sem fio: uma pessoa sussurra uma frase para quem está ao lado, que repassa o melhor que consegue para a próxima pessoa da fila, e assim por diante, até a última pessoa dizer em voz alta o que entendeu. Quase sempre a frase final é uma versão distorcida da original, às vezes irreconhecível.

Por trás dessa brincadeira está um resultado central da teoria da informação, que também está relacionado a sistemas de inteligência artificial que processam dados em etapas sucessivas:

uma câmera fotografa uma cena, um algoritmo comprime a imagem, uma rede neural extrai características em camadas cada vez mais abstratas, até chegar a um diagnóstico ou uma decisão final. Será que, em algum ponto desse processo, alguma etapa poderia encontrar informação nova sobre o dado original que as etapas anteriores não tinham captado? A desigualdade de processamento de dados responde a essa pergunta e esta atividade serve para vocês comprovarem sozinhos, com simulações, por que a resposta é não.

2.1 Bloco 1: Conceitos principais

Cadeia de Markov. Três variáveis aleatórias X, Y, Z formam uma cadeia de Markov, denotada $X \to Y \to Z$, quando Z é condicionalmente independente de X dado Y, isto é,

$$p(z \mid y, x) = p(z \mid y) \quad \Longleftrightarrow \quad I(X; Z \mid Y) = 0.$$

Intuitivamente, toda a influência de X sobre Z passa necessariamente por Y. No jogo do telefone sem fio, cada pessoa só ouve o que a pessoa anterior disse, não a frase original. É exatamente essa a estrutura de uma cadeia de Markov.

Desigualdade de processamento de dados. Se $X \to Y \to Z$ é uma cadeia de Markov, então

$$I(X;Y) \;\geq\; I(X;Z),$$

usando a informação mútua definida no Tópico 1 (equação (2)). Quanto mais processamento, quanto mais elos entre a fonte original X e a variável final Z, menos informação sobre X pode restar. Isso vale mesmo que o processamento seja extremamente sofisticado: nenhuma função aplicada a Y, por mais elaborada que seja, consegue recuperar informação sobre X que já tenha sido perdida na etapa anterior. Em particular, para qualquer função, determinística ou aleatória, $Z = g(Y)$, vale $I(X;Y) \geq I(X;g(Y))$.

Por que isso importa para inteligência artificial. Todo sistema de aprendizado de máquina que processa uma entrada em etapas, seja uma imagem passando por camadas de uma rede neural, um texto sendo tokenizado e depois embutido em vetores, ou um modelo grande sendo destilado em um modelo menor, é, sob o ponto de vista da teoria da informação, uma cadeia de Markov. A desigualdade de processamento de dados garante um teto absoluto: nenhuma camada de uma rede neural pode conter mais informação sobre a entrada original, ou sobre o rótulo verdadeiro, dependendo de como a cadeia é montada, do que a camada anterior continha. Essa ideia está por trás da teoria do "gargalo de informação"(information bottleneck) para redes profundas, que vamos discutir mais para frente. Está relacionado com um problema recente e bastante discutido: quando um modelo generativo é treinado repetidamente sobre "dados sintéticos" em vez de dados reais, cada geração é mais um elo na cadeia a partir da distribuição real original, e a informação sobre essa distribuição real só pode se degradar geração após geração.

2.2 Bloco 2: Roteiro de simulação

Passo 1: Escolham uma variável fonte X representando uma mensagem original, por exemplo um único bit, ou um pequeno vetor de bits.

Passo 2: Modelem cada etapa de processamento como um canal ruidoso simples, por exemplo um canal binário simétrico que inverte cada bit de forma independente com probabilidade p, aplicado sucessivamente. Gerem Y_1 a partir de X passando por esse canal, depois Y_2 a partir de Y_1 passando por outro canal, e assim por diante, formando a cadeia $X \to Y_1 \to Y_2 \to \cdots \to Y_k$.

Passo 3: Simulem essa cadeia inteira para N mensagens independentes, gerando N valores de X e propagando cada um pela cadeia completa, obtendo amostra suficiente para estimar as distribuições conjuntas necessárias.

Passo 4: Estimem $I(X;Y_1), I(X;Y_2), \ldots, I(X;Y_k)$ a partir das amostras simuladas, e verifiquem numericamente que a sequência é aproximadamente não crescente, dentro da margem de erro esperada pela estimação com amostra finita.

Passo 5: Façam um gráfico dessa sequência de informações mútuas contra a posição na cadeia, e observem visualmente a degradação da informação, elo após elo.

Passo 6: Repitam o experimento variando a intensidade do ruído em cada elo, canais mais ou menos ruidosos, e observem como isso afeta a velocidade de degradação da informação ao longo da cadeia.

Passo 7: Construam agora uma versão inspirada em um processo de inteligência artificial real: tratem X como um rótulo verdadeiro, por exemplo duas classes, gato e cachorro, gerem uma imagem simplificada, um vetor de números cuja distribuição depende do rótulo, apliquem uma etapa de compressão, reduzindo a dimensão do vetor, por exemplo mantendo só algumas coordenadas ou fazendo médias de blocos de coordenadas, para obter uma representação intermediária, e finalmente apliquem uma regra de decisão simples sobre essa representação para produzir uma predição final.

Passo 8: Calculem a informação mútua entre o rótulo verdadeiro e cada estágio do processo, imagem completa, representação comprimida, predição final, e verifiquem novamente a desigualdade de processamento de dados ao longo dessas três etapas.

Passo 9: Para investigar o colapso de modelo, simulem um processo de várias gerações: a partir de uma distribuição real conhecida, gerem uma amostra (geração 0), estimem uma distribuição aproximada a partir dela, gerem uma nova amostra a partir dessa distribuição aproximada (geração 1), estimem uma nova distribuição aproximada a partir dela, e repitam esse processo por várias gerações. Meçam como a informação mútua, ou a divergência, entre os dados de cada geração e a distribuição real original evolui ao longo das gerações.

2.3 Bloco 3: Sugestões de implementação em Python

Reaproveitem as funções de entropia e informação mútua construídas no Tópico 1. A cadeia de canais ruidosos do Passo 2 é simples de vetorizar: representem cada estágio como um vetor `numpy` de zeros e uns, apliquem a inversão de bits sorteando, para cada posição, se ocorre erro, comparando um vetor de números aleatórios uniformes com a probabilidade p do canal, e combinem esse vetor de erros com o estágio anterior por meio de uma operação `XOR` (ou-exclusivo). Guardem cada estágio da cadeia, não só o último, em uma lista, já que vocês vão precisar calcular a informação mútua de X com cada estágio, não somente com o final.

Para o processo de inteligência artificial do Passo 7, não é preciso usar nenhuma biblioteca de aprendizado profundo. Uma forma simples e transparente de gerar a imagem é usar vetores gaussianos cuja média depende da classe, uma média para gato, outra para cachorro, mais ruído gaussiano independente em cada coordenada. A etapa de compressão pode ser uma projeção simples em menos coordenadas, e a etapa final de decisão pode ser só uma comparação com um limiar, por exemplo a distância a cada uma das duas médias. Se algum grupo quiser ir além, o `scikit-learn` tem classificadores simples prontos, como regressão logística ou árvore de decisão, para essa última etapa, mas o ponto pedagógico do exercício não depende disso.

Para a simulação de gerações do Passo 9, representem a distribuição como um vetor de probabilidades sobre um número pequeno de categorias, cinco ou dez já devem bastar para o efeito

aparecer claramente. A cada geração, sorteiem uma amostra de tamanho finito a partir da distribuição atual, estimem as novas probabilidades a partir dessa amostra, a própria frequência relativa já serve, e usem essa nova estimativa como a distribuição da geração seguinte. Vale a pena repetir o experimento com tamanhos de amostra diferentes por geração, para ver se o efeito piora quando cada geração aprende a partir de poucos exemplos. Acompanhem a divergência $D(p_{\text{real}} \| p_{\text{geração } k})$ ao longo das gerações, reaproveitando a função de divergência do Tópico 1, e guardem todas as gerações, não só a última, para poderem "plotar"(*urgh!...*) a trajetória completa.

Um cuidado geral nesta atividade: como vocês vão calcular várias informações mútuas em sequência ao longo de uma cadeia, pequenos erros de estimação em cada elo se acumulam. Se a curva de informação mútua que vocês obtiverem não for perfeitamente decrescente, algum solavanco para cima entre um elo e o seguinte, antes de desconfiar da teoria, aumentem o tamanho da amostra N e vejam se o solavanco desaparece. É provavelmente ruído de estimação, não uma violação da desigualdade. Shannon agradece...

3 Tópico 3: Compressão de Dados, AEP e o Critério de Kelly

O cenário

Imaginem uma casa de apostas esportivas que oferece cotações para os jogos de uma temporada de futebol inteira. Cada partida tem três resultados possíveis: vitória do time da casa, empate, ou vitória do time visitante. Se preferir pense no valor de certa ação na bolsa de valores em uma determinada data que pode subir, descer ou ficar estável. Duas aplicações diferentes, (ambas me parecem repugnantes) que ilustram o mesmo modelo matemático.

Um apostador tem seu próprio palpite sobre as probabilidades reais de cada resultado, possivelmente melhor informado que a casa, possivelmente pior, e precisa decidir, partida após partida, que fração do seu patrimônio apostar em cada resultado. Apostar demais é arriscado, uma sequência de azar pode quebrar o apostador. Apostar de menos "deixa dinheiro na mesa", mesmo tendo uma vantagem real.

Existe uma estratégia matematicamente ótima para esse dilema, descoberta por John Kelly em 1956 nos laboratórios Bell, os mesmos onde Claude Shannon trabalhava. Esta atividade mostra o que ela tem a ver com entropia.

3.1 Bloco 1: Conceitos principais

Propriedade de equipartição assintótica (AEP). Para uma sequência $X_1, \ldots, X_n$ de variáveis aleatórias independentes e identicamente distribuídas segundo p, a lei dos grandes números aplicada a $-\log_2 p(X_i)$ garante que

$$-\frac{1}{n} \log_2 p(X_1, \ldots, X_n) \xrightarrow{P} H(X).$$

Na prática, isso significa que quase toda a probabilidade se concentra em um conjunto de sequências "típicas", cada uma com probabilidade aproximadamente $2^{-nH(X)}$, e esse conjunto típico tem aproximadamente $2^{nH(X)}$ sequências, mesmo que o número total de sequências possíveis seja muito maior. É essa concentração que torna a compressão de dados possível: basta um código eficiente para as sequências típicas, o resto praticamente nunca acontece.

O jogo de apostas. Considerem m resultados possíveis, o resultado i ocorrendo com probabilidade verdadeira p_i, com a casa de apostas pagando "o_i para 1" (uma aposta de uma unidade monetária no resultado i retorna o_i unidades caso i ocorra, e nada caso contrário). Um apostador

que distribui frações b_i do seu patrimônio entre os resultados, com $\sum_i b_i = 1$, vê seu patrimônio evoluir ao longo de n apostas independentes segundo

$$S_n = S_0 \prod_{j=1}^{n} b_{X_j}\, o_{X_j}.$$

A taxa de crescimento exponencial típica do patrimônio é

$$W(b,p) = \mathbb{E}\big[\log_2(b_X\, o_X)\big] = \sum_i p_i \log_2(b_i o_i).$$

O critério de Kelly. A escolha de b que maximiza $W(b,p)$, entre todas as distribuições de aposta possíveis, é simplesmente $b_i^* = p_i$: apostem de acordo com as probabilidades verdadeiras dos resultados, não de acordo com as cotações oferecidas pela casa. Chamando $W^* = W(p,p)$ essa taxa ótima, no caso particular de cotações uniformes "justas" $o_i = m$ para todo i, uma casa ingênua que paga igual para todos os resultados ignorando que eles não são igualmente prováveis, a taxa ótima se reduz a

$$W^* = \log_2 m - H(p).$$

Quanto menor a entropia da distribuição verdadeira, quanto mais previsível o resultado, maior a vantagem que um apostador bem informado consegue extrair de uma casa que não percebe esse padrão.

O custo de acreditar no modelo errado. Se o apostador usa a estratégia de Kelly baseada em uma distribuição q que ele acredita ser verdadeira, mas a distribuição real dos resultados é $p \neq q$, a taxa de crescimento efetivamente obtida, $W(q,p)$, fica abaixo da ótima exatamente pela divergência entre as duas distribuições,

$$W^* - W(q,p) = D(p\|q),$$

a mesma entropia relativa do Tópico 1, equação (1), agora com uma interpretação bem concreta: excesso de confiança em crenças erradas custa dinheiro, e custa exatamente $D(p\|q)$ bits de taxa de crescimento por aposta. Vale notar que essa igualdade não depende de quais sejam as cotações o_i, ela é sempre verdadeira!

Informação lateral. Se, antes de cada aposta, o apostador recebe uma dica Y correlacionada com o resultado real X, um informante, uma notícia sobre a escalação do time, uma lesão de última hora, o aumento na taxa ótima de crescimento obtido ao usar essa informação da melhor forma possível é

$$\Delta W = I(X;Y),$$

exatamente a informação mútua entre o resultado e a dica, também independente das cotações oferecidas. Nenhuma quantidade de informação lateral pode gerar um ganho maior que isso, e um apostador suficientemente esperto conseguiria extrair esse ganho.

3.2 Bloco 2: Roteiro de simulação

Passo 1: Definam o mundo real da simulação: o número m de resultados possíveis por partida, por exemplo $m = 3$, as probabilidades verdadeiras p_i de cada resultado, escolhidas por vocês, e as cotações o_i oferecidas pela casa de apostas para cada resultado.

Passo 2: Gerem uma longa sequência de n resultados de partidas, independentes e identicamente distribuídos segundo as probabilidades verdadeiras p_i. Essa é a temporada simulada.

Passo 3: Implementem três estratégias de aposta: apostar tudo sempre no resultado mais provável, distribuir a aposta igualmente entre todos os resultados, e a estratégia de Kelly, apostando a fração $b_i = p_i$ em cada resultado.

Passo 4: Para cada estratégia, simulem a evolução do patrimônio ao longo da temporada inteira, partida a partida, partindo de um patrimônio inicial comum às três.

Passo 5: Repitam esse experimento completo, a temporada inteira de n apostas, muitas vezes, digamos 500 ou 1000 temporadas simuladas de forma independente, para cada estratégia, guardando o patrimônio final de cada repetição.

Passo 6: Comparem as três estratégias observando a taxa de crescimento típica, a média entre as repetições de $\frac{1}{n}\log_2$ do patrimônio final, e também o quão frequentemente cada estratégia leva a um patrimônio final próximo de zero ou a ganhos extraordinários.

Passo 7: Comparem a taxa de crescimento empírica de cada estratégia com o valor teórico $W(b, p)$ previsto pela fórmula do Bloco 1.

Passo 8: Para observar a propriedade de equipartição assintótica em ação, façam um histograma de $\frac{1}{n}\log_2$(patrimônio final), obtido nas muitas repetições do Passo 5, para alguns valores crescentes de n, por exemplo $n = 10, 100, 1000$, e observem como a distribuição fica cada vez mais concentrada ao redor do valor teórico W^* conforme n cresce.

Passo 9: Introduzam agora uma informação lateral: antes de cada aposta, o apostador recebe uma dica Y, uma variável correlacionada com o resultado real da partida mas imperfeita. Definam a distribuição conjunta de resultado e dica, implementem a estratégia de Kelly condicional a essa informação, apostando $b_i(y) = p(i \mid y)$ a cada rodada dependendo da dica recebida, e meçam o aumento na taxa de crescimento obtido.

Passo 10: Comparem esse ganho empírico com a informação mútua entre resultado e dica, calculada diretamente da distribuição conjunta definida no Passo 9.

Passo 11: Por fim, simulem um apostador que acredita, erradamente, em probabilidades q diferentes das verdadeiras p, por exemplo superestimando as chances do time favorito, e que aposta de acordo com Kelly usando q em vez de p. Comparem a taxa de crescimento resultante com a taxa ótima e verifiquem se a perda bate com a divergência $D(p\|q)$.

3.3 Bloco 3: Sugestões de implementação em Python

Sorteiem os resultados da temporada com `numpy.random.choice`, passando o vetor de probabilidades verdadeiras. Para a evolução do patrimônio, trabalhem em log em vez de multiplicar o patrimônio diretamente a cada rodada: somem os incrementos $\log_2(b_{X_j} o_{X_j})$ ao longo do tempo, por exemplo com uma soma acumulada, e só exponenciem no final, se precisarem do valor do patrimônio em si. Multiplicar o patrimônio centenas ou milhares de vezes seguidas pode facilmente estourar os limites numéricos do computador, para cima ou para baixo, enquanto a soma de logaritmos não tem esse problema.

Para simular muitas temporadas de uma vez de forma eficiente, pensem em uma matriz de resultados com uma linha por temporada e uma coluna por rodada, dimensão repetições por n,

gerada de uma só vez, em vez de um laço em Python repetindo a simulação de uma temporada de cada vez. Operações vetorizadas do `numpy` sobre essa matriz inteira vão ser muito mais rápidas.

Para a informação lateral do Passo 9, representem a distribuição conjunta de resultado e dica como uma pequena matriz $m \times k$ de probabilidades. Como vimos em aula, um jeito direto de sortear pares (X, Y) consistentes com essa conjunta é achatar a matriz em um único vetor de probabilidades sobre as $m \times k$ combinações, sortear um índice desse vetor achatado, e depois recuperar os valores de X e Y correspondentes à posição sorteada na matriz original. A partir da mesma matriz conjunta, as distribuições condicionais $p(x \mid y)$ usadas na aposta condicional são apenas as colunas da matriz, normalizadas para somar 1.

Guardem o patrimônio final, ou melhor, o log do patrimônio final, de cada repetição em um vetor ou em uma coluna de um `DataFrame` do `pandas`, uma coluna por estratégia, para poder comparar médias, medianas e histogramas facilmente com `matplotlib` ao final.

4 Tópico 4: Capacidade de Canal e Códigos Corretores de Erro

O cenário

Uma sonda espacial pousa em Marte e começa a enviar fotos de volta para a Terra. O sinal de rádio percorre mais de 200 milhões de quilômetros, chega extremamente fraco, e se mistura com ruído térmico dos próprios equipamentos e ruído cósmico de fundo. Cada bit enviado tem uma certa probabilidade de ser recebido invertido, um 0 virando 1 ou vice-versa. A energia disponível na sonda é escassa, painéis solares pequenos e empoeirados, baterias limitadas, então não é uma opção simplesmente transmitir mais forte para vencer o ruído. A agência espacial precisa de uma forma de garantir que as fotos cheguem sem erros, ou com uma taxa de erro tão baixa quanto necessário, mesmo por um canal ruidoso e com pouca energia disponível.

Claude Shannon provou, em 1948: é possível transmitir informação com uma taxa de erro tão pequena quanto se queira, contanto que a taxa de transmissão fique abaixo de um limite fundamental do canal, chamado capacidade. Esta atividade simula esse cenário e coloca vocês para testar, na prática, os limites e as promessas do teorema da codificação de canal de Shannon, do mesmo jeito que a NASA precisou fazer de verdade nas missões Mariner e Voyager.

4.1 Bloco 1: Conceitos principais

Canal binário simétrico. O modelo mais simples de canal ruidoso: cada bit transmitido é recebido corretamente com probabilidade $1 - p$ e invertido com probabilidade p, de forma independente para cada bit. É talvez um modelo razoável para o elo de rádio Terra-Marte, com p representando a intensidade do ruído.

Capacidade do canal. A capacidade $C = \max_{p(x)} I(X;Y)$, o máximo sendo tomado sobre todas as distribuições de entrada possíveis e usando a informação mútua do Tópico 1, equação (2), é o maior número de bits de informação, em média, que se pode transmitir de forma confiável por uso do canal. Para o canal binário simétrico com parâmetro p,

$$C = 1 - H_b(p), \qquad H_b(p) = -p \log_2 p - (1-p) \log_2 (1-p),$$

alcançada quando os bits transmitidos são igualmente prováveis, metade zeros e metade uns. Quando $p = 0$, canal perfeito, a capacidade é 1 bit por uso do canal. Quando $p = 0.5$, o canal é pura moeda ao acaso e o bit recebido não tem relação nenhuma com o enviado, a capacidade cai a zero.

Teorema da codificação de canal (Shannon). Para qualquer taxa de transmissão $R < C$, medida em bits de informação úteis por bit efetivamente transmitido pelo canal, existem

esquemas de codificação, com blocos de comprimento suficientemente grande, que alcançam probabilidade de erro tão próxima de zero quanto se queira. Para $R > C$, ao contrário, nenhum esquema de codificação, por mais engenhoso que seja, consegue manter a probabilidade de erro afastada de zero. A capacidade é uma fronteira real, uma *transição de fase*, não uma limitação de engenharia contornável com mais criatividade.

Redundância como proteção. Um código que transforma k bits de informação em um bloco de $n \geq k$ bits transmitidos tem taxa $R = k/n$. Bits extras, a redundância, permitem ao receptor detectar e corrigir erros introduzidos pelo canal, ao custo de transmitir mais bits para a mesma quantidade de informação útil. Dois exemplos clássicos que vocês vão implementar: o código de repetição, que simplesmente repete cada bit várias vezes e decide por votação majoritária no receptor, e o código de Hamming $(7, 4)$ (comentado nas "aulas de motivação" do curso), que transforma 4 bits de informação em um bloco de 7 bits, 3 deles de paridade, que resulta capaz de corrigir qualquer erro de um único bit no bloco.

4.2 Bloco 2: Roteiro de simulação

Passo 1: Modelem o canal de comunicação Terra-Marte como um canal binário simétrico com probabilidade de erro p, escolhendo um valor que deixe os efeitos bem visíveis nas simulações, algo talvez entre 0.05 e 0.2.

Passo 2: Calculem a capacidade teórica desse canal, $C = 1 - H_b(p)$, em bits por uso do canal.

Passo 3: Gerem uma sequência longa de bits de informação, pensem neles como pixels de uma imagem simplificada, cada um 0 ou 1, que representa a foto a ser enviada para a Terra.

Passo 4: Simulem a transmissão direta, sem nenhuma codificação de proteção: cada bit passa pelo canal ruidoso e pode ser recebido invertido. Meçam a taxa de erro de bit, a fração de bits recebidos incorretamente, e comparem com o valor teórico p.

Passo 5: Implementem um código de repetição: cada bit de informação é enviado como um bloco de n_r cópias idênticas, por exemplo 3 ou 5, e o receptor decide o valor do bit original por votação majoritária entre as cópias recebidas, usando sempre n_r ímpar para não ter empates. Calculem a taxa efetiva desse código e meçam a taxa de erro de bit depois da decodificação.

Passo 6: Implementem o código de Hamming $(7, 4)$: a cada 4 bits de informação, calculem 3 bits de paridade segundo as regras do código, formem um bloco de 7 bits, transmitam pelo canal ruidoso, e no receptor usem os bits de paridade recebidos para detectar e corrigir um eventual erro de 1 bit no bloco. Calculem a taxa desse código e meçam a taxa de erro de bit resultante.

Passo 7: Reúnam, em uma mesma tabela ou gráfico, a taxa de erro de bit e a taxa efetiva das três abordagens, sem codificação, repetição, Hamming, para o mesmo valor de p, e comentem qual delas oferece o melhor equilíbrio entre proteção contra erros e uso eficiente do canal.

Passo 8: Repitam o experimento para diferentes valores de p, canal mais ou menos ruidoso, e observem como a vantagem de cada esquema de codificação muda.

Passo 9: Para investigar a promessa do teorema de Shannon, aumentem o comprimento do

bloco do código de repetição, $n_r = 3, 5, 7, 9, \ldots$, calculando a cada vez a taxa efetiva correspondente, e observem se a taxa de erro de bloco realmente cai conforme o comprimento aumenta. Relacionem essa queda com a proximidade entre a taxa efetiva do código e a capacidade C calculada no Passo 2.

Passo 10: Escolham por fim um cenário em que a taxa de transmissão de informação desejada é maior que a capacidade do canal, $R > C$, e tentem, com os códigos à disposição de vocês, reduzir a taxa de erro a um nível aceitável. Documentem o que acontece e relacionem o resultado com a parte do teorema de Shannon que afirma ser isso impossível, para qualquer código.

4.3 Bloco 3: Sugestões de implementação em Python

Representem os bits como vetores `numpy` de inteiros 0 e 1, ou booleanos. O canal binário simétrico se implementa em poucas linhas: gerem um vetor de números aleatórios uniformes do mesmo tamanho da mensagem, comparem com p para decidir em quais posições ocorre inversão, e combinem esse vetor de erros com a mensagem original por `XOR`.

Para o código de repetição, reorganizem o vetor de bits de informação em blocos, por exemplo com `numpy.reshape`, repetindo cada bit n_r vezes antes de reorganizar, apliquem o canal ruidoso ao vetor todo de uma vez, e depois decodifiquem somando os valores dentro de cada bloco de n_r posições e comparando com $n_r/2$.

Para o código de Hamming $(7, 4)$, procurem, nas notas de aula, em um livro de teoria da codificação, ou consultem seu "AI assistant" favorito, a matriz geradora 4×7 e a matriz de verificação de paridade 3×7 padrão desse código, e implementem a codificação e a decodificação como operações de álgebra linear módulo 2, multiplicação de matrizes seguida do resto da divisão por 2, no lugar da soma e multiplicação usuais. A decodificação por síndrome, calcular o produto entre a matriz de verificação e o bloco recebido módulo 2 e usar o resultado para localizar a posição do erro, é elegante mas exige atenção aos detalhes de convenção de qual bit é qual. Testem a codificação e a decodificação com um único bloco e um erro inserido manualmente em uma posição conhecida antes de rodar a simulação completa, para garantir que a lógica está correta.

Ao medir taxas de erro pequenas, tomem cuidado com o tamanho da amostra: para estimar com confiança razoável uma taxa de erro da ordem de p, é preciso observar da ordem de $100/p$ bits ou blocos transmitidos, para que ocorram ao menos umas boas dezenas de eventos de erro na simulação. Com poucos bits transmitidos, a taxa de erro medida pode facilmente dar zero só por acaso, mesmo que a taxa verdadeira não seja desprezível, e vocês tirariam uma conclusão errada sobre a qualidade do código.

Para comparar com a teoria, a probabilidade de erro de bloco do código de repetição, a probabilidade de que a maioria das n_r cópias esteja errada, é uma cauda de distribuição binomial, calculável diretamente com `scipy.stats.binom`, uma conferência simples da simulação. Ao final, um gráfico da taxa de erro em escala logarítmica, que costuma deixar mais clara a diferença entre os esquemas, contra o comprimento do bloco ou contra o tipo de código, com uma linha horizontal de referência marcando a taxa de erro sem codificação, comunica bem os resultados no relatório.